

When 01:30 happens twice

Daily learning note | 26 September 2026 | Q005a

Sidiq review status: Not yet reviewed.

Research question and hypothesis

Can a timezone-aware availability selector still admit information from the future when local clocks move backwards? The hypothesis is that comparing UTC instants for every timestamp fixes the known London fold leak while preserving publication, revision, duplicate, latency and staleness rules.

Q002 exposed this defect in Q001 on 22 September. Q004 established that real data access remains unverified. Repairing the selector is the next independent prerequisite before Q005 ingestion; it needs no market subscription and makes no empirical investment claim.

Beginner explanation and terms

A wall clock is a label for time, not a unique position on a timeline. When clocks move backwards, a label can occur twice. In our London example, the first 01:30 has offset +01:00 and the second has offset +00:00. A report arriving at the second occurrence was unavailable at the first.

Instant: one position on the timeline. **UTC:** the common reference used here for chronology. **Offset:** the difference between a local clock and UTC. **Named zone:** a location's time rules, such as Europe/London. **Fold:** the marker distinguishing the earlier and later repeated time. **Latency:** elapsed processing time after publication. **Age:** elapsed time since observation.

A timezone-aware object contains enough information to locate an instant, but the chosen comparison operation still matters. Using UTC consistently also gives duplicate detection a common identity.

Data, metrics and protocol declared before implementation

All observations are invented. The central case uses London 25 October 2026; the regression grid also uses London 29 March and New York 8 March and 1 November 2026. These are clock-transition fixtures under installed timezone rules, including future fixture dates, not observed research activity.

Acceptance: reject the one-hour future publication; accept latency at exact equality; reject stale records despite fresh revisions; select by observation instant then revision instant; distinguish two fold releases and reject the same release expressed in another zone. Existing UTC results must remain unchanged. An integer-minute oracle, boundary cases and two identical experiment runs provide checks. There is no fitted model, tuning search, return series or alpha estimate.

Mathematics and worked example

Let o , a and d be observation, availability and decision instants expressed in UTC. Let $L \geq 0$ be processing latency and $M \geq 0$ an optional age limit, in elapsed time units. A valid record has $a \geq o$. Eligibility requires $a \leq d$ and $d - a \geq L$, plus $d - o \leq M$ when M is supplied. For each asset and feature, select the eligible maximum pair (o, a) , comparing o first, then a . A revision changes a and the value; it does not reset o or make an old observation young again.

For this example $\text{UTC} = \text{local wall time} - \text{offset}$. Conversion must preserve the instant; attaching a UTC label without changing the clock fields changes what the timestamp means.

Event on 25 October 2026	London label and offset	UTC instant
Earlier decision	01:30, +01:00, fold=0	00:30
Later publication	01:30, +00:00, fold=1	01:30

At the earlier decision, $d - a = -60$ minutes: the later publication must be excluded. The old raw same-zone predicate instead treated the two local labels as equal and selected it. At the later decision, a record published at the first 01:30 has waited 60 minutes, so $L = 60$ passes. An observation from that first instant has age 60 minutes, so $M = 59$ rejects it even after revision.

At the spring transition, London 00:30 to 02:30 on 29 March spans only 60 elapsed minutes. A 90-minute latency must therefore fail, while a 90-minute age limit still passes. Both transitions matter: local arithmetic can understate or overstate elapsed time.

Primary sources and design decision

Python's datetime documentation states that comparison ignores fold when operands share the same tzinfo object, and subtraction in that case does no offset adjustment. Its astimezone operation preserves the UTC instant. We use that conversion before validation and selection.

<https://docs.python.org/3/library/datetime.html>

PEP 495 defines the earlier/later fold distinction and explains that constructors permit invalid local-time instances, including times inside a spring-forward gap. UTC conversion cannot recover an unspecified source fold or prove that a supplied local clock label existed. <https://peps.python.org/pep-0495/>

Python's zoneinfo documentation describes fold-aware conversion and its use of system IANA data or the tzdata package. The fixture explicitly records offsets so changed rules cause a visible test failure rather than a silent change in expected chronology. <https://docs.python.org/3/library/zoneinfo.html>

These primary sources were consulted on 26 September 2026. Canonical UTC storage is our design choice, consistent with Q002's label intervals, rather than a claim that all applications require it.

Implementation and reproducible commands

The Observation constructor now validates awareness, converts observed_at and available_at to UTC, checks their order and stores the converted timestamps. snapshot converts decision_at before doing any comparisons. Existing selection, duplicate and elapsed-duration logic then uses one timeline. The original Observation object is returned; its source zone label is no longer stored on the record. Caller datetime objects remain unchanged. Runtime dependencies are unchanged.

The reusable change is in src/quant_research/data/availability.py. Regression cases live in tests/test_availability_timezones.py. Run make fold-experiment for the worked counterexample; results are in experiments/timezones/results.json. The deliberately invalid comparator uses raw local inputs to show the old predicate; it is not a second production selector.

Run make check for tests, lint, formatting, strict type checking and the C++ output check. Run make experiment to reproduce Q001, and make manifest-experiment for the existing Q003 audit. Run make note NOTE_DATE=2026-09-26 to rebuild this PDF.

Observed results and validation

Check	Observed result
Future publication at earlier fold	Legacy admits it; corrected selector rejects it
Local versus elapsed difference	0 local minutes; 60 elapsed minutes
Latency 60 / age limit 59	Exact latency accepted; stale observation rejected
Latest revision	Value 2 selected, independent of input order
Before-fix regression run	16 failed, 6 passed; experiment test excluded
Full Python suite after fix	167 passed, including 23 new cases
Quality and dependencies	Ruff, 27 formatted files, mypy and pip consistency passed
C++ and prior artifacts	Warning-clean build and golden output; Q001/Q003 unchanged

The grid compares 10,800 snapshot results against an independent integer-minute schedule across four transitions, three record-zone representations, 25 decisions, two latencies, three age limits, three decision zones and two input orders. Exact boundary tests also vary thresholds by one microsecond. Duplicate tests cover equivalent instants in different zones even before publication becomes eligible. The new experiment ran twice in separate processes and matched the committed JSON byte-for-byte.

The original defect was reproduced before editing production code. All 16 observed regression failures were resolved. The full check passed on its first post-fix run. Restricted Git fetch initially failed DNS; approved fetch succeeded. GitHub's authenticated issue read returned no open issues. No coverage percentage or remote CI result is inferred from local checks.

Limitations and interview explanation

UTC normalization does not verify source clocks, publication truth, calendar sessions, permissions or historical vintages. It accepts the instant implied by an aware input. Ambiguous strings need an explicit source policy; nonexistent local times need ingestion validation. Leap seconds are outside this model. Named-zone tests need an IANA timezone database; no new tzdata dependency was added today.

This changes stored timestamp representation: consumers expecting a London zone label must retain that metadata upstream or convert explicitly for display. Equal instants now share a canonical duplicate key; the two distinct fold instants remain distinct. There is no measured speedup or trading-performance claim.

For an interview, draw the two UTC instants first, then show their identical local labels. Explain why awareness alone did not make the original operations chronological. Walk through the publication and age inequalities, show a failing test, and distinguish this fix from source-data validation.

Next work and required input

Q005 ingestion is next and should define identifier, duplicate, source-zone and invalid-local-time policies. Q004a remains blocked on lawful data access and sample evidence; source or budget input is needed only before real acquisition. C002 remains the Wednesday 30 September C++ step. Sunday synthesis can replan original target dates from actual progress. No personal review is required to continue engineering work.

Check your understanding

1. Why can two aware London timestamps showing 01:30 represent different instants?
2. At the second 01:30, does a publication from the first 01:30 satisfy a 60-minute latency?
3. Can a new revision make a 60-minute-old observation pass a 59-minute age limit?
4. Why are equivalent UTC and local representations duplicates, while two fold releases are not?
5. Does UTC normalization prove that an ambiguous timestamp was assigned the correct fold?

Suggested answers

1. Their offsets differ. In this fixture fold=0 is 00:30 UTC and fold=1 is 01:30 UTC. The local clock repeats a label while elapsed time continues forwards.
2. Yes. Exactly 60 elapsed minutes have passed and the contract includes equality. A threshold one microsecond greater would fail, even though local subtraction misleadingly reports zero.
3. No. Age is measured from the original observation instant. A revision updates availability and value, not the observation's age.
4. Duplicate identity uses asset, feature, observation instant and availability instant. Alternate zone representations keep those instants unchanged; two folds differ by one hour in this fixture.
5. No. Conversion preserves the instant implied by the supplied input. Choosing the right fold, rejecting nonexistent local times and verifying source evidence remain ingestion responsibilities.